

---

BY BRIAN HOUCK, MAX KANAT-ALEXANDER, EIRINI KALLIAMVAKOU,  
MARGARET-ANNE STOREY, AND NICOLE FORSGREN

---

# CAFE(S): Your Agent Is Only As Good As Its Context

*Five durable properties for evaluating context quality*

Ask an engineering leader how to get more value from AI coding agents, and the conversation usually turns to models. They'll talk about newer models, larger context windows, stronger reasoning, or more capable agent architectures. That focus is understandable; models supply much of the visible intelligence in an AI system.

But organizations are increasingly discovering that model capability alone does not determine the result. The same model can perform very differently depending on the information it receives.<sup>1</sup> Give an agent an unclear objective, incomplete requirements, outdated architectural guidance, or irrelevant documentation, and it may spend its time (and tokens) making a plausible but incorrect assumption or confidently produce the wrong result. Give it a clear task, relevant supporting information, and appropriate constraints, and the outcome can be dramatically better.

This is not an entirely new problem. Software organizations have long struggled to document decisions, preserve institutional knowledge, keep information current, and help people find what they need. Knowledge management,<sup>7</sup> information retrieval,<sup>22</sup> requirements engineering,<sup>32</sup> and technical communication all address important parts of that challenge. AI agents do not make those disciplines obsolete. In many ways, they expose the cost of neglecting them.

What is new is the role that assembled context now plays in the engineering system. An agent may draw from source code, specifications, issue trackers, chat histories, architectural documents, tool output, and user instructions, then act on that combination with limited human supervision. Information that was written for different audiences, at different times, and for different purposes is brought together into a temporary operating environment for a particular task. Whether each source is individually useful does not reveal whether the resulting context is fit for the work the agent is being asked to do.

That creates a gap between several established concerns. Knowledge management asks whether useful information exists and is maintained. Retrieval systems determine whether that information can be found. Context engineering focuses on selecting and assembling it for a model. Less clearly defined is how to evaluate the quality of the assembled context itself. Is it clear enough to interpret? Does it support action? Does it accurately represent current reality? Is it focused enough to be useful? Is it safe and appropriate for the agent to consume?

These questions matter, because software engineering is becoming a collaborative system in which a human developer works together with agents to deliver software. When the working context is poor, developers compensate by locating missing information, clarifying requirements, correcting misunderstandings, and recovering from avoidable errors. When it is good, agents

become more reliable collaborators, and developers can spend more of their effort on the engineering judgment that requires them most.

This article proposes CAFE(S), a framework for describing the quality of context assembled for an AI system. CAFE(S) is organized around five dimensions: clarity, actionability, fidelity, efficiency, and security. It does not prescribe a retrieval architecture or a particular method of context engineering. Instead, it provides a shared language for asking whether the resulting context gives a human-agent system the conditions it needs to work effectively. The parenthetical S reflects that security complements the other four dimensions by asking a different question about the same context. We return to that distinction later.

CAFE(S) is deliberately a definition for high-quality context; it is not a measurement system. The goal of this article is to describe these properties clearly enough that teams can discuss them, review context for them, and intervene when they are missing. A clearer definition may also create the foundation for future work to measure context quality and examine its relationship to agent performance, developer experience, and software outcomes.

The sections that follow explain why context quality has become an operational concern, define what is meant by context and where it sits in the larger AI system, describe each CAFE(S) property, examine common failure patterns, and offer practical guidance for individuals and teams.

## Why Context Quality Matters Now

AI does not create a new need for good knowledge management; it dramatically increases the cost of poor knowledge management. Context has always mattered. What has changed is not its importance, but that its consequences are now experienced immediately, at scale, and in ways that are often directly measurable. Poor context now carries a bill you can read on an invoice, a liability you can be sued over, and a drag on developer experience that compounds every day.

Three forces—developer experience, cost, and liability—have turned what was once an abstract organizational concern into a first-order engineering problem.

**Developer Experience** The most common cost of poor context is also the least visible, because it appears as friction rather than a line item. When context is unclear, an agent guesses. When it is incomplete, the agent produces something plausible but wrong. When it is stale, the agent confidently applies a policy that no longer exists. Every one of those failures lands on a developer, who now has to notice the problem, diagnose it, re-explain what they wanted, and try again.

The collaboration has broken down, not because either participant lacks capability, but because they were not operating from the right information. Developers have always spent time compensating for poor knowledge, documentation, and communication. AI changes the scale and frequency of those interactions by making context the input to nearly every task an agent performs. The agent is supposed to remove toil, but poor context hands it back.

This is why context quality is not simply an infrastructure concern or an optimization for AI systems. It is a direct input to developer experience.<sup>9</sup> Healthy context creates the conditions under which humans and agents can collaborate effectively. Poor context forces humans to spend their time compensating for avoidable mistakes.

**Cost** Context is also where the economics of agentic software become visible. Every token sent to an agent carries a cost, and context is where those tokens accumulate. When Notion engineered its context for reuse through prompt caching, its costs dropped by roughly 90 percent while maintaining output quality, because reading cached context costs substantially less than repeatedly transmitting the same information.<sup>2</sup> Relevance pays as well as reuse. When Meta precomputed concise context files for a data pipeline spanning multiple repositories and more than 4,100 files, agents completed tasks using roughly 40 percent fewer tool calls and tokens.<sup>12</sup>

Organizations that repeatedly send large, redundant, or poorly structured context are paying more than organizations that engineer context to be lean, reusable, and task-specific. As agents tackle longer-running workflows and more autonomous tasks, the financial consequences of context quality grow alongside them.

**Liability** Poor context also creates organizational risk. Whether the underlying failure is stale information, incomplete context, or hallucination, organizations remain accountable for what their agents communicate. In February 2024, the British Columbia Civil Resolution Tribunal found Air Canada liable for a refund its customer-service chatbot promised but the company's actual policy did not provide.<sup>6</sup> Air Canada argued that the chatbot was effectively responsible for its own statements. The tribunal disagreed, holding that the company remained accountable for information presented through its chatbot just as it would be for information on any other part of its website. The damages were modest. The precedent was not.

The stakes rise further when the context itself is compromised. In June 2025, researchers disclosed EchoLeak (CVE-2025-32711), the first documented prompt-injection attack to achieve real-world data exfiltration from a production AI system.<sup>29</sup> A single crafted email, never opened by the user, caused Microsoft 365 Copilot to retrieve sensitive internal documents into its working context and leak their contents. Months later, CamoLeak (CVE-2025-59145) demonstrated the same class of attack against GitHub Copilot Chat using hidden instructions embedded within a pull request.<sup>24</sup>

Neither system was “hacked” in the traditional sense. Untrusted information simply became part of the agent's context, and the agent faithfully acted upon it. If the context is compromised, the reasoning built on top of it will be compromised as well.

This broader concern also connects to prior work on a Triple Debt Model by Storey.<sup>30</sup> The model argues that, in addition to technical debt, a project may accumulate cognitive debt when a team's understanding of the system's behavior, structure, and history is never fully established or gradually erodes over time.

It also describes intent debt, which is the erosion of explicit rationale, goals, and constraints that are needed by developers and agents to evolve the system effectively over time. CAFE(S) may be useful in identifying when intent debt is manifesting to the point that the context provided to the agents is insufficient and causing operational issues. An organization can carry intent debt invisibly for years, but the moment someone delegates work to an agent, only the knowledge present in the assembled context remains available. Whatever intent was never made explicit is simply absent.

None of this suggests that better context is a cure-all or that a well-provisioned agent can safely operate without oversight. The claim is narrower, and we believe, more useful. Context quality has become a first-order determinant of what agentic software costs, how safely it behaves, and whether humans can rely on it as an effective collaborator. That is reason enough to define what good context actually is.

## What Is Context?

The rapid rise of AI engineering has introduced a growing vocabulary: prompt engineering, retrieval-augmented generation (RAG), context engineering, harness engineering, agent frameworks, memory systems, and model evaluation. Each addresses a different part of building effective AI systems, yet they all assume a more fundamental question that has received comparatively little attention.

To answer the question of what makes context “good” first requires being precise about what is meant by context.

At its most basic level, context is the set of information available to an agent while it performs a task. For a language model, it is the collection of tokens considered during inference. It is the instructions it receives, the task description, code and documentation, retrieved knowledge, tool outputs, memory from previous interactions, and any other information presented before generating a response. Everything an agent produces is ultimately a function of that context.

Context engineering is the practice of deciding what belongs in that set. As the field has matured, attention has shifted from prompt engineering—crafting individual instructions—to the broader challenge of assembling, maintaining, and updating the right information throughout an agent's workflow. Anthropic describes context engineering as ensuring the optimal set of tokens during inference.<sup>1</sup> Context engineering is fundamentally a practice. It describes how context is assembled; it does not tell us whether the resulting context is actually good.

Before context quality can be evaluated, more fundamental knowledge-management concerns must already have been addressed. Information must exist, be accessible, be connected to the agent, and be discoverable through retrieval. These have been central concerns of knowledge management and information retrieval for decades,<sup>7,22</sup> and AI amplifies rather than replaces them. CAFE(S) deliberately begins after those prerequisites have been satisfied. It asks how to evaluate quality once the relevant information has been assembled into an agent's working context.

To understand where that missing question fits, it helps to separate the major layers of an AI system.

### **Intent → Context → Harness → Model**

This simple pipeline describes how human intent becomes machine behavior.

Intent is what a person ultimately wants to accomplish. That intent must then be translated into context, which is the information that represents the task, its constraints, and the surrounding knowledge. A harness applies that context through orchestration, retrieval, tools, memory, and execution logic before passing it to a model, which supplies the underlying intelligence that produces a result.

Each layer has become the focus of its own engineering discipline. Advances in machine learning improve models. Agent and harness engineering improve how those models are orchestrated.<sup>4</sup> Product and software engineering improve how human intent is captured, refined, and communicated. Yet relatively little work has focused on the translation between intent and the context that captures and represents it, despite that translation increasingly determining whether even highly capable agents succeed or fail.

CAFE(S) deliberately focuses on one layer of that pipeline: the translation from intent to context. It is a framework neither for evaluating models nor for designing harnesses. Instead, it provides a way to reason about whether the context handed to an agent faithfully represents what a human is actually trying to accomplish.

**What Makes Context Good?** Before handing anything to an agent, you can ask what a capable person would need to do the same task well. They would need to understand the goal, have the relevant background, know the constraints that apply, and have some way to recognize when the work is correct. An agent needs much the same information, usually in much the same form. Rather than thinking of good context as an exotic new artifact, you can think of it as what a competent colleague would need to do the job, if that colleague could not easily lean over and ask a follow-up question.

Good context is defined by what it enables. Context is good when it lets a human-agent system accomplish its intended task correctly, efficiently, and safely. It gives the agent enough information to reason instead of guess, grounds its work in current and trustworthy information, keeps its attention on what matters, and avoids exposing information it should never have received. With poor context, agents compensate by guessing, reasoning from stale or conflicting information, wasting effort searching through noise, or acting on information they should never have been given.

Because good context is ultimately defined by the results it produces, CAFE(S) does not replace that definition; instead, it answers the narrower and more useful question of what context tends to look like when it reliably leads to good outcomes. The framework in the next section is our answer to that question. Rather than prescribing how context should be assembled, CAFE(S) names five enduring properties that separate healthy context from unhealthy—the characteristics most likely to produce good outcomes—and gives teams a shared language for evaluating the information that agents are asked to work from.

## **The CAFE(S) Framework**

CAFE(S) names five properties of good context: clarity, actionability, fidelity, efficiency, and security (table 1). The first four describe whether context lets an agent do good work; the fifth, set slightly apart in parentheses, describes whether that context is safe to use at all. Together they form a

checklist you can hold a piece of context against, the way you might hold a pull request against a set of review criteria. They allow you to ask if any given context is good enough to hand to an agent.

**Table 1. The CAFE(S) framework. Key dimensions, core definitions, and evaluation questions for high-quality context in AI agent execution. Five properties of high-quality context.**

| Clarity <b>SPECIFIC &amp; NON-AMBIGUOUS</b>      | Actionability <b>DIRECTIVE &amp; GOAL ORIENTED</b> | Fidelity <b>ACCURATE &amp; UP TO DATE</b>      | Efficiency <b>SCOPED &amp; TOKEN OPTIMIZED</b> | Security <b>SAFE &amp; COMPLIANT</b>       |
|--------------------------------------------------|----------------------------------------------------|------------------------------------------------|------------------------------------------------|--------------------------------------------|
| Can the agent interpret the request as intended? | Can the agent proceed, and know when it is done?   | Can the agent trust that this context is true? | Can the agent focus on what matters?           | Should the agent have this context at all? |

The properties are meant to be largely independent, with each dimension naming a distinct way context can succeed or fail. Context can be clear but not faithful, describing the system as it was, rather than as it is. It can be faithful and secure but not efficient, burying the one detail that matters. It can be efficient and faithful but not actionable, never saying what finishing looks like. Independence across the dimensions does not mean a given failure maps to exactly one pillar. A single bad instruction can fail several at once, but each pillar isolates a dimension you can reason about, review for, and fix on its own.

Each property has a definition, the question it answers, and the characteristic way it breaks. Throughout the framework, the properties of context are deliberately separated from the mechanisms that create them and the outcomes they influence. This keeps the definitions stable even as engineering practices, tools, and models continue to evolve.

**Clarity: Can the Agent Interpret the Request as Intended?** Clarity is the property of context being unambiguous for the task at hand and for the party that has to act on it. Clear context resolves the same way for the author and the agent. Unclear context admits several interpretations, and the agent has no reliable way of knowing which reading was intended. Context need not be unambiguous in some absolute sense, only in ways that affect the work. A term the task never touches can stay loose. A term the task turns on is different. If the context does not settle which system, environment, or policy is meant, the agent will settle it for you, plausibly and silently.

The practical consequence of poor clarity is avoidable variation. When context is up for interpretation, different executions may produce different outcomes. This is not because the task itself changed or because of the normal variability inherent in probabilistic models, but because the context left important decisions open to interpretation.

Clarity failures are easy to miss because they are invisible to the author. The person who created the context usually knows what they meant, so the ambiguity is visible only to a reader or agent who does not already share their intent or their conventions. Teams develop local vocabularies, and a term that is precise inside one group can admit several readings outside it. Language, domain, and cultural convention all shape what counts as an obvious reading, and an agent shares none of them by default.<sup>12</sup>

When context is clear, agents spend less time interpreting requests and more time solving the intended problem.

**Actionability: Can the Agent Proceed and Know When it is Done?** Actionability is the property of context supplying enough direction to act. That means describing the objective, the constraints that bear on it, and a definition of what “done” looks like. Actionable context lets an agent proceed without guessing and recognize when the task is complete. Context that lacks actionability leaves the agent working toward a target that was never fully specified.

Where clarity asks whether the context can be read one way, actionability asks whether enough was said. The two are independent. A request can be perfectly unambiguous and still leave out the goal, say too little about the constraints, or give no way to tell that the work is finished.

Poor actionability has three practical consequences: (1) The agent may never really start, spending its effort reconstructing an objective that was never stated; (2) it may never stop, because nothing in the context defines what finished looks like; or (3) it may stop in the wrong place, declaring success against a target it inferred rather than one it was given. The third is the most expensive, because the work looks like progress.

Actionability is easy to underspecify because the author is closest to the work. When the goal and the finish line are already settled in your own head, stating them feels redundant, so they go unwritten, and the agent is left with nothing to go on.

**Fidelity: Can the Agent Trust That the Context is True?** Fidelity is the property of context being accurate. Is it a faithful representation of the system, the organization, and the task the agent is being asked to perform? Context with poor fidelity may contain outdated documentation, conflicting instructions, unsupported assumptions, or information from unreliable sources. An agent reasoning over inaccurate context may produce something that appears internally consistent while being fundamentally wrong because it was grounded in an incorrect understanding of reality.

Fidelity also covers what the context does not say. A gap rarely stays a gap, often forcing an agent to make a guess that then becomes indistinguishable from fact.

Maintaining fidelity is particularly challenging because the systems, organizations, and goals that context describes evolve continuously.<sup>30</sup> Code changes, APIs are deprecated, architectural decisions are revisited, and organizational policies shift over time. Context that was once an accurate representation of reality can quietly become stale, leading agents toward obsolete implementations or invalid assumptions.

This is why fidelity is inseparable from verification. The question is not only whether the context is accurate today, but whether there is a mechanism for keeping it accurate as the system evolves. Source-of-truth documents, freshness checks, attribution, and regular review all help preserve fidelity over time.

**Efficiency: Is the Context Lean Enough to be Usable and Affordable?** Efficiency is the property of context maximizing signal while minimizing irrelevant information. Efficient context gives the agent what matters without burdening it with information that is unnecessary. It matters for two reinforcing reasons. The first is that irrelevant context does not merely consume space; it competes with relevant information for the model's attention, making it harder to reason effectively.<sup>20</sup> The second reason is cost. Every unnecessary token is paid for, and that cost compounds across every session.

You ask about one small piece of a system and hand the agent the whole codebase anyway. Almost everyone does this, and most know they are doing it. Working out what is relevant takes effort. Including everything takes none. Efficiency is the discipline of resisting that, and it is worth stressing that it is not brevity for its own sake. The goal is not the smallest possible context, but the highest signal-to-noise ratio.<sup>12,21</sup> Every token that competes for attention without helping the task should go.

**Security: Is the Context Safe and Compliant to Use?** The first four properties ask whether context enables an agent to do good work. Security asks a more fundamental question: Should the agent have been given this context in the first place? It belongs alongside the other dimensions because it evaluates the same artifact, the assembled context, even though it asks a different question about it. Failures of the first four dimensions make agents less effective. Failures of security make them unsafe. Sensitive information can be exposed, policies can be violated, or an attacker may manipulate the agent through the context it receives when this dimension fails.

Two broad classes of failure dominate. The first is untrusted context. Information retrieved from sources an attacker can influence may carry instructions, or simply false assertions, intended to steer the agent. Model defenses against this are improving, but whether a given model can defend the attack is not the point. The point is whether attacker-controlled content was in the context in the first place. This is the mechanism behind the exfiltration attacks described earlier, and it echoes the lesson software engineers learned from SQL injection. Information cannot be trusted simply because it arrives through a legitimate channel.

The second is overshared context. Agents can receive secrets, personal data, or regulated information they do not actually need to perform the task. Once included in the working context, that information may be logged, cached, transmitted, or surfaced in ways the organization never intended. Security therefore concerns both keeping malicious information out of an agent's context and keeping unnecessary sensitive information out.

**Why These Five** A fair question is why the framework stops at five when there are other candidates such as timeliness, cost, provenance, and accessibility. Our test is that a property earns its own pillar only if it is not derivable from the others, can be acted on independently, and names a failure the others miss. Most candidates fold cleanly into an existing pillar. Stale context is simply context that is no longer true, making timeliness a facet of fidelity. Cost is a downstream consequence of efficiency, not a separate property of the artifact. Provenance is how you keep context trustworthy, which falls under fidelity.

Accessibility is intentionally outside the scope of CAFE(S). Whether information can be located and retrieved is a prerequisite concern of knowledge management and retrieval systems. CAFE(S) assumes that the relevant information has already been assembled and evaluates the quality of that assembled context.

These five are not the only defensible cuts, and we expect edge cases. But these dimensions are independent, actionable, and, collectively, they sufficiently cover the ways context succeeds or fails without redundant overlap.

## When Context Goes Wrong

One useful way to understand the CAFE(S) dimensions is to look at what happens when they break down. Poor context does not produce random failures. The same patterns appear repeatedly across different models, tools, and organizations. Naming those patterns gives teams something concrete to review for and discuss, much like software engineers use terms such as code smell to describe recurring design problems. In fact, you could think of these failure patterns as context smells.

The examples that follow are illustrative rather than exhaustive, and each primarily highlights one or more CAFE(S) dimensions.

**Confident Hallucination (Fidelity)** Hallucinations are often described as model failures, but they are just as often failures of fidelity. A model can distinguish supported claims from unsupported ones only if the surrounding context gives it a way to do so. Lawyers have now been sanctioned for submitting briefs that cited cases that never existed.<sup>23</sup> The immediate cause was a model hallucination, but the deeper failure was that nothing in the workflow treated those citations as claims requiring verification. When confidence substitutes for evidence, fabricated information becomes difficult to distinguish from fact.

**Zero-Click Leak and Silent Exfiltration (Security)** Security failures often begin when agents cannot distinguish trusted information from untrusted instructions. In the previously mentioned EchoLeak, a crafted email that was never opened by the user caused Microsoft 365 Copilot to retrieve internal information and leak it.<sup>29</sup> In the previously mentioned CamoLeak, hidden instructions embedded in a pull request caused GitHub Copilot Chat to expose secrets from private repositories.<sup>24</sup> In both cases, the attack succeeded because hostile content became part of the agent's working context. The boundary between data and instructions disappeared.

**Weekend Runaway (Efficiency + Actionability)** Give an agent a task without a definition of success and it may never discover that it is finished. Instead, it retries, expands scope, or continues exploring an objective that cannot actually be completed. The result is wasted time, unnecessary token consumption, and sometimes surprisingly large bills after an agent spends hours or days pursuing an underspecified goal.<sup>5</sup> The problem is both one of efficiency, because resources are consumed without bound, and one of actionability, because the context never defined what "done" looked like.

**Lost in the Middle (Efficiency)** More context is not always better. Model performance degrades as input length grows, and it does so unevenly. What matters is not only whether the required information is present, but also how much competing material surrounds it.<sup>5,20</sup> Nothing is technically missing. The agent has access to the required information, yet it still fails because the

signal has been overwhelmed by noise. This illustrates why efficiency is not simply about reducing token costs. A smaller, more focused context can produce better results than a larger one.

**Specification Ambiguity (Clarity)** Faced with an obviously underdetermined request, an agent may stop and ask for clarification. Faced with a semi-ambiguous request, agents generally do something much more dangerous. They choose an interpretation and proceed.<sup>19</sup> If that initial assumption is wrong, every subsequent step compounds the error. The failure is not that information is missing, but that the existing context admits more than one reasonable interpretation.

**Impossible Task (Actionability)** Agents are often asked to do things that cannot actually be done. Rather than recognizing that no solution exists, they may continue searching, retrying, or inventing increasingly implausible approaches because nothing in the context established the boundaries of what was achievable or what success would look like.<sup>5</sup>

**Contradictory and Fragmented Context (Clarity + Efficiency)** Adding more context does not necessarily make an agent better informed.<sup>18</sup> Two documents may disagree about how a system behaves. A design document may conflict with the implementation. Critical information may exist but only when pieces are assembled from multiple repositories, documents, and conversations that are never viewed together. The raw information exists, yet the assembled context lacks a coherent picture of reality. The agent spends effort reconciling inconsistencies or searching for missing connections instead of solving the problem it was asked to address.

Table 2 summarizes these failure patterns, together with additional recurring context failures and the CAFE(S) dimensions they primarily violate.

**Table 2. Common agent failure patterns and affected context dimensions. A mapping of runtime agent failure modes to their corresponding CAFE(S) dimensions.**

| Failure Pattern                             | Brief Description                                                                                                                                                               | Dimensions                 |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| Confident Hallucination <sup>23</sup>       | The model generates incorrect information and presents it with unwarranted confidence, making errors difficult to detect.                                                       | Fidelity                   |
| Zero-Click Leak <sup>29</sup>               | Untrusted content is automatically processed and influences model behavior without explicit user action.                                                                        | Security                   |
| Silent Exfiltration <sup>24</sup>           | Malicious instructions embedded in retrieved content cause an agent to expose sensitive information.                                                                            | Security                   |
| Weekend Runaway <sup>5</sup>                | Poor constraints or stopping criteria lead an agent to consume excessive tokens, time, or money.                                                                                | Efficiency + Actionability |
| Lost in the Middle <sup>15,20</sup>         | Critical information exists in context but is overlooked because of placement, volume, or retrieval limitations.                                                                | Efficiency                 |
| Specification Ambiguity <sup>19</sup>       | Multiple reasonable interpretations of the request lead the agent to construct an incorrect understanding of the intended task and proceed under that assumption.               | Clarity                    |
| Impossible Task <sup>5</sup>                | Telling the model to do something that can't be done results in the agent spiraling into illogic and unsuccessful loops.                                                        | Actionability              |
| Contradictory Context <sup>18</sup>         | Multiple sources within the context provide conflicting information, forcing the model to arbitrate between incompatible truths.                                                | Clarity + Fidelity         |
| Stale Guidance <sup>12,21</sup>             | Context accurately reflects a previous state of reality but is no longer current, resulting in incorrect decisions or guidance.                                                 | Fidelity                   |
| Cross-Channel Eavesdrop <sup>27</sup>       | Information from one communication channel unintentionally affects reasoning or exposes information originating in another.                                                     | Security                   |
| Paste-in-the-Prompt Leak <sup>28</sup>      | Sensitive data is unintentionally disclosed by users pasting confidential information into external AI systems.                                                                 | Security                   |
| Schema Drift <sup>14</sup>                  | The expected structure of tool inputs, outputs, or retrieved data evolves over time, causing downstream failures even though individual components continue behaving correctly. | Fidelity                   |
| Negative-Constraint Fragility <sup>17</sup> | Context emphasizes what the agent must not do without sufficiently specifying what it should do instead.                                                                        | Actionability              |
| Indirect Prompt Injection <sup>13</sup>     | Retrieved content contains instructions that compete with or override the agent's intended task.                                                                                | Security                   |

| Failure Pattern                     | Brief Description                                                                                                                                                  | Dimensions                 |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| Goal Drift <sup>3</sup>             | An agent gradually optimizes for a proxy objective or interpretation of the task that differs from the user's original intent.                                     | Actionability              |
| Lost in the Details <sup>30</sup>   | Context supplies implementation detail without the rationale behind it, leaving the agent to optimize local mechanics while losing track of what the task was for. | Actionability + Efficiency |
| Referential Ambiguity <sup>19</sup> | Multiple entities satisfy the same reference, causing the agent to reason about or modify the wrong object despite correctly following the request.                | Clarity                    |
| Overspecification <sup>21</sup>     | Context prescribes how a task should be done in more detail than it warrants, displacing behavior the model would have performed better unguided.                  | Efficiency + Actionability |

Taken together, these examples illustrate an important point. In many cases, the necessary information was already in the assembled context. The problem was that the information was stale, ambiguous, contradictory, buried in noise, or mixed with content that should never have been trusted in the first place. Asking whether an agent has some context is therefore the wrong question. The more useful question is whether the assembled context is actually fit for the task at hand. That is precisely the question the CAFE(S) framework is intended to help answer.

# What Good Context Looks Like in Practice

Before a team can worry about whether its context is clear, actionable, faithful, efficient, and secure, it has to clear a more basic set of hurdles.

The first precondition is simply accessibility. The most common context problem today is not that context is low quality but that the agent cannot reach it. The sources exist, but they have not been wired into the agent, often because of real security constraints or the ordinary difficulty of change inside a large organization. Context that exists but is unreachable helps no one.

The second precondition is cutting through the noise. Once the sources are connected, the agent has to find the relevant slice. If 100 Slack channels share a prefix, which should it read, and how much can it read before it exhausts the window or the budget? A keyword search may suffice in simple cases, but what if the needed context lives across many systems of record rather than the one being worked in?

The third precondition is the hardest: The fact the context has to exist at all. A great deal of organizational knowledge has never been written down—it lives in hallway conversations, unrecorded meetings, and private messages an agent cannot (and often should not) see. No amount of context engineering can retrieve what was never captured.<sup>12</sup>

Only after those three hurdles are cleared do the CAFE(S) dimensions become the operative constraint. This is worth stating plainly because it reframes where most teams should start. Not with a scoring rubric but with the unglamorous work of connecting sources, making them searchable, and writing down what only lives in people's heads.

Importantly, “good” does not mean perfect. Context should be good enough for the task at hand. A slightly verbose context is usually preferable to one that is missing critical information or that contains inaccurate guidance. In practice, teams should first eliminate missing, incorrect, and stale context before optimizing for elegance or brevity.

**Context Quality is Relative to Where It Lives** One principle cuts across everything that follows: The quality of a piece of context depends on where it lives and when it is used. The same instruction can be excellent in one location and harmful in another.

Consider an AGENTS.md file at the root of a repository—a file whose contents are loaded into every agent session. An instruction about how to run the test suite may belong there, because nearly every session needs it. However, an instruction about one rarely touched file does not—it will be injected into thousands of sessions that have nothing to do with that file, spending tokens and attention on noise.<sup>21</sup> That same instruction, placed as a comment inside the file it describes, is suddenly high quality—clear, efficient, and available exactly when relevant. Nothing about the words changed; only their location did. Where context lives is one of its most important properties, and evaluating context means evaluating it in situ, not in the abstract.

**What Individuals Can Do** The highest-leverage move for an individual is to be rigorous about the human-written text they hand to agents and to recognize that creating and maintaining good context is itself part of software engineering. It is not something done after the “real work” is finished. A prompt is context. But so is a code comment, an architecture document, a README, a checked-in specification. Any of these can be read by an agent as instruction or as fact, and any of them, if stale or ambiguous, can send the agent down a wrong path—an expensive one. An architecture document that no longer matches the code will not just mislead the agent; it will burn tokens as the agent chases files that no longer exist. So, the discipline is to treat everything you write that an agent might read as being subject to the same standards you would apply to the context itself. Is it clear, is it current, is it scoped to where it lives, does it avoid telling the agent to trust something it should not?

The second move for an individual to make is to write good context deliberately, the way a senior engineer briefs a junior one. This is not the arcane “prompt engineering” of a few years ago; it is a straightforward discipline of stating what you are trying to accomplish, how you would know it was solved, what is off limits and why, where the relevant code lives (if you are confident in it), and under what circumstances the agent should come back to you. Notably, this does not require elaborate prompt templates or systems that force developers into a rigid format—those tend to lose their value as models improve. The durable investment is educating engineers in the craft of writing clear, honest, well-scoped context, the same way they have been taught to write readable code and good comments.

**What Teams Can Do** Individual rigor scales into team practice through shared, consistent sources of context. Specifications stored as code, AGENTS.md files, checked-in skills and examples, architecture documents, shared glossaries, and the other artifacts a whole team relies on are all important practices. Maintaining those artifacts should be treated as ordinary software maintenance. Like tests, build systems, or code itself, shared context degrades unless teams continuously invest in it.

Because context is shared, its defects are amplified. A stale architecture document or an ambiguous AGENTS.md file is not read by one developer; it is read by every agent session that consumes it. That is why one of the highest-leverage team practices is a context-review discipline. Just as code review catches defects early and reinforces shared standards, reviewing important context artifacts against the CAFE(S) dimensions before they are broadly reused catches ambiguity, staleness, unnecessary complexity, and unsafe instructions before they propagate.

Not every artifact requires the same level of scrutiny, however. A repository-wide AGENTS.md file that shapes thousands of sessions deserves far more review than a working note shared between two engineers. Review effort should be proportional to the impact context has when it is reused.

Teams should also establish conventions for where different kinds of context belong. Because context quality depends on where it is consumed, agreeing where design documents, operational runbooks, architectural decisions, coding conventions, and repository-wide instructions live makes it easier for both humans and agents to discover and maintain them. The goal is not rigid standardization for its own sake but creating predictable places where important knowledge can be found.

There is a balance to strike, though. Not all context should be forced through public review or shared repositories. A great deal of it is genuinely personal, such as how someone likes to work, their own priorities, or notes to themselves, and publishing all of it would both reduce individual productivity and pollute everyone else’s context with irrelevant information. The team’s responsibility is to identify the context that should be shared, reviewed, and maintained as common knowledge.

**What Organizations Can Do** At the organizational level, the challenge expands beyond individual repositories to infrastructure, education, policy, and long-term stewardship. Organizations need the infrastructure that allows agents to retrieve information safely from companywide systems, the governance that prevents sensitive or malicious context from being introduced into agent workflows, and clear ownership so important context does not quietly decay as systems and teams evolve.

Every major shared context source—runbooks, architecture documents, policies, glossaries, schemas, and platform guidance—should have a clear owner and an expected review cadence so

fidelity does not silently erode over time. Ownership should follow teams rather than individuals wherever possible, ensuring that important context survives reorganizations and personnel changes. Likewise, retrieval infrastructure should make high-quality context discoverable while keeping restricted information appropriately protected.

Organizations should also invest in education. Writing and maintaining good context is an engineering skill, much like writing maintainable code or conducting effective code reviews. Engineers should understand not only how to work effectively with agents, but also why context quality matters and how the CAFE(S) dimensions can be applied when creating or reviewing shared context.

Finally, organizations should recognize context as an engineering asset rather than a by-product of feature delivery. Capturing architectural rationale, documenting important decisions, and maintaining shared understanding all require time, yet they compound in value as both humans and agents increasingly rely on them. Organizations that reward only short-term feature delivery will watch their shared context gradually decay, while those that invest in maintaining high-quality shared context will create healthier environments for both human developers and AI agents.

**Good Context Enables Autonomy—It Does Not Guarantee It** It would be easy to read this framework as a checklist that, once satisfied, clears an agent for autonomy. It does not. Healthy context is a prerequisite for trusting an agent with more autonomy, but it is not a guarantee of it. Clear, actionable, faithful, efficient, and secure context creates the conditions under which higher autonomy becomes reasonable; whether to grant that autonomy still depends on the stakes of the task, the organization’s tolerance for risk, and where accountability sits when something goes wrong. CAFE(S) describes what good context is. It does not claim that good context, by itself, is sufficient to hand an agent the keys—only that without it, you should not.

## Where CAFE(S) Fits with What You Already Use

An engineering leader reading this has a reasonable question: I already measure delivery, I run retrieval evaluations, I follow security guidance, and my platform team has an observability stack—do I need this too? The answer is yes, and the reason is that none of those tools evaluates what CAFE(S) does. They explore adjacent parts of the system; CAFE(S) describes the quality of the context artifact itself.

The distinctions are easiest to see side by side, as shown in table 3.

**Table 3. Comparative analysis of CAFE(S) and established frameworks. Scope mapping and incremental capabilities provided by CAFE(S) relative to existing engineering and AI evaluation frameworks.**

| What you may already use                                              | What it covers                                                                            | What CAFE(S) adds                                                                                                           |
|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| DORA <sup>11</sup>                                                    | The throughput and stability of the software delivery system                              | Evaluates the information environment feeding that system, where delivery problems increasingly originate                   |
| SPACE, <sup>10</sup> DX Core 4, <sup>25</sup> EngThrive <sup>16</sup> | The human side of productivity, and experience—and the outcomes teams are trying to reach | Evaluates the quality of the shared information environment that increasingly shapes both human and agent effectiveness     |
| RAG evaluation frameworks <sup>8</sup>                                | Retrieval and generation quality—largely fidelity, partly efficiency                      | Unifies those with clarity, actionability, and security in one quality frame, rather than treating retrieval in isolation   |
| OWASP LLM Top 10 <sup>26</sup>                                        | LLM application security risks in depth                                                   | Locates security (the S pillar) inside a broader quality model, so safety is evaluated alongside usefulness, not separately |
| Context- and harness-engineering                                      | How to construct good context and harnesses—prescriptive design                           | Provides the criteria for evaluating the quality of the resulting context, regardless of how it was constructed.            |

| What you may already use                                            | What it covers                                                                                   | What CAFE(S) adds                                                                                                                       |
|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| guidance (Anthropic, <sup>1</sup><br>Böckeler, <sup>4</sup> others) |                                                                                                  |                                                                                                                                         |
| Triple Debt Framework <sup>30</sup>                                 | System-level software health over time through the lens of technical, cognitive, and intent debt | Names where intent debt becomes concrete and immediate                                                                                  |
| TRUCE <sup>31</sup>                                                 | How developers and managers define software quality, in their own words, across five dimensions  | Applies the same concept of defining artifact quality across dimensions, but to the properties that make context good, rather than code |

The pattern throughout table 3 is that CAFE(S) is orthogonal and complementary. It is not a replacement for any of these; it is a quality scorecard for the context artifact designed to sit on top of the stack you already run. It runs on your observability tooling rather than against it, and it evaluates the context that standards such as the Model Context Protocol (MCP) help you assemble—MCP standardizes how context is gathered, while CAFE(S) asks whether what is gathered is any good.

The relationship worth examining more closely is the one with Storey’s work,<sup>30</sup> because it is the closest in spirit. Where the triple debt model describes intent debt as the erosion of the goals and rationale a system needs to evolve safely, it also describes cognitive debt as the erosion of the team’s understanding, structure, and history of the project.

CAFE(S) focuses on a point at which these longer-term forms of debt may become operationally visible. When a task is delegated to an agent, the agent can draw only on the context made available to it. Intent that was never externalized or that has become outdated, fragmented, or misaligned over time may therefore be absent or poorly represented, while cognitive debt may limit the team’s ability to recognize and correct those deficiencies. In this sense, CAFE(S) offers a way to assess the immediate context in which cognitive and intent debt may have practical consequences rather than measuring either form of debt directly.

# Conclusion

It is worth returning to the chain this article started with:

**Intent → Context → Harness → Model**

The model supplies raw intelligence, and it is improving at a pace that dominates the headlines. The harness applies that intelligence through orchestration, tools, and memory, and it is the focus of intense engineering effort. Both matter enormously. Yet both are ultimately bounded by the quality of the context they receive. No matter how capable the model or how sophisticated the harness, an agent can reason only from the information available to it.

CAFE(S) complements established disciplines such as knowledge management, information retrieval, and context engineering. Those disciplines determine whether the right information can be found and assembled. Instead, CAFE(S) asks a narrower question of whether the assembled context is actually good enough for an agent to do its best work.

Our claim is intentionally modest. We argue that context quality can be described in terms of five durable properties—clarity, actionability, fidelity, efficiency, and security—and that naming those properties gives teams a shared language for reasoning about context as a first-class engineering artifact. Like code, context can be reviewed, owned, improved, and allowed to decay. Unlike code, however, it has largely escaped systematic evaluation despite increasingly determining how effectively humans and agents work together.

We have also been deliberate about scope. This article defines a framework; it does not introduce a measurement system. Future work will be needed to develop reliable ways of assessing these properties empirically and to understand how improvements in context quality influence collaboration, software delivery, and organizational outcomes. Meaningful measurement requires a shared definition of what is being measured. Before teams can improve context quality, they need a vocabulary for discussing it.

The broader implication extends beyond any particular model, framework, or generation of AI systems. Models will continue to improve. Harnesses will become more sophisticated. The

tools and techniques of context engineering will continue to evolve. The need for context that is clear, actionable, faithful, efficient, and secure will not. Those qualities are properties of effective communication, not of any specific technology.

As software engineering increasingly becomes a collaborative activity between humans and AI agents, the quality of that collaboration will depend on not only the intelligence of our models, but also the quality of the information environment created for them. Improving human-agent collaboration therefore requires more than building smarter agents. It requires building healthier contexts. Treating context as a first-class engineering artifact and giving teams a common language for reasoning about its quality is an important step toward that future.

## Related Material

### **DevEx: What Actually Drives Productivity**

The developer-centric approach to measuring and improving productivity.

Abi Noda Margaret-Anne Storey, Nicole Forsgren, Michaela Greiler

<https://queue.acm.org/detail.cfm?id=3595878>

### **DevEx in Action**

A study of its tangible impacts

Nicole Forsgren, et al.

<https://queue.acm.org/detail.cfm?id=3639443>

### **Assessing IT Project Success: Perception vs. Reality**

We would not be in the digital age if it were not for the recurrent success of IT projects.

João Varajão and António Trigo

<https://queue.acm.org/detail.cfm?id=3687999>

## Acknowledgments

We would like to thank Collin Green, Josh Beitler, Autumn Faust and Grace Fu, along with all of the other reviewers who helped shape this article in its final form.

COPYRIGHT © 2026 HELD BY OWNER/AUTHOR. PUBLICATION RIGHTS LICENSED TO ACM. This work is licensed under a Creative Commons Attribution International 4.0 License.

---

## References

1. Anthropic. Effective context engineering for AI agents. Anthropic. 2025; <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>.
2. Anthropic. Case study: Notion builds agent orchestration where teams and Claude collaborate on real work. Claude by Anthropic. 2026; <https://claude.com/customers/notion>.
3. Arike, R., Donoway, E., Bartsch, H., and Hobbhahn, M. Technical Report: evaluating goal drift in language model agents. 2025. arXiv 2505.02709; <https://arxiv.org/abs/2505.02709>.
4. Böckeler, B. Harness engineering for coding agent users. martinowler.com. 2026; <https://martinowler.com/articles/harness-engineering.html>.
5. Cemri, M. et al. Why do multi-agent LLM systems fail? 2025. arXiv 2503.13657; <https://arxiv.org/abs/2503.13657>.
6. Civil Resolution Tribunal British Columbia Moffatt v. Air Canada, 2024 BCCRT 149 (CanLII). 2024; <https://canlii.ca/t/k2spq>.
7. Davenport, T.H. and Prusak, L. *Working Knowledge: How Organizations Manage What They Know*. Harvard Business Review Press, Boston, MA 1998.
8. Es, S., James, J., Espinosa Anke, L., and Schockaer, S. RAGAs: automated evaluation of retrieval augmented generation. *Proceedings of the 18th Conf. of the European Chapter of the Association of the Computational Linguistics: System Demonstrations*, 2024. 150–158; <https://aclanthology.org/2024.eacl-demo.16/>.

9. Forsgren, N. et al. DevEx in action. *Communications of the ACM* 67, 6 2024, 42–51; <https://dl.acm.org/doi/10.1145/3643140>.
10. Forsgren, N. et al. The SPACE of developer productivity: There's more to it than you think. *acmqueue* 19, 1 2021, 20–48; <https://queue.acm.org/detail.cfm?id=3454124>.
11. Forsgren, N., Humble, J., and Kim, G. *Accelerate: The Science of Lean Software and DevOps—Building and Scaling High Performing Technology Organizations*. IT Revolution Press, Portland, OR 2018.
12. Ganerwal, K., Rath, P., and Verma, A. How Meta used AI to map tribal knowledge in large-scale data pipelines. Engineering at Meta. 2026; <https://engineering.fb.com/2026/04/06/developer-tools/how-meta-used-ai-to-map-tribal-knowledge-in-large-scale-data-pipelines/>.
13. Greshake, K. et al. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. 2023. arXiv 2302.12173; 10.48550/arXiv.2302.12173.
14. Hasan, M. StructHallu-Drift: Benchmarking structured hallucinations under schema evolution in LLMs. *Proceedings of the First Workshop on Structured Understanding, Retrieval, and Generation in the LLM Era (SURGeLLM)* Association for Computational Linguistics 2026, 333–343; <https://aclanthology.org/2026.surgetlm-1.22/>.
15. Hong, K., Troynikov, A., and Huber, J. Context rot: how increasing input tokens impacts LLM performance. Chroma technical report. 2025; <https://research.trychroma.com/context-rot>.
16. Houck, B., Bozarth, T., Liu, D., and Carignan, D. EngThrive: Make it fast and easy to do great work. *acmqueue* 24, 3 2026; <https://queue.acm.org/detail.cfm?id=3819080>.
17. Jang, J., Ye, S., and Seo, M. Can large language models truly understand prompts? A case study with negated prompts. *Proceedings of the First Transfer Learning for Natural Language Processing Workshop, Proceedings of Machine Learning Research* 203, 2023. 52–62; <https://proceedings.mlr.press/v203/jang23a.html>.
18. Li, J., Raheja, V., and Kumar, D. ContraDoc: understanding self-contradictions in documents with large language models. *Proceedings of the 2024 Conf. of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies 1* (Long Papers), 2024, 6509–6523; <https://aclanthology.org/2024.naacl-long.362/>.
19. Liu, A. et al. We're afraid language models aren't modeling ambiguity. *Proceedings of the 2023 Conf. on Empirical Methods in Natural Language Processing* Association for Computational Linguistics, 2023, 790–807; <https://aclanthology.org/2023.emnlp-main.51/>.
20. Liu, N.F. et al. Lost in the middle: how language models use long contexts. *Transactions of the Association for Computational Linguistics* 12, 2024, 157–173; [https://direct.mit.edu/tac/article/doi/10.1162/tac\\_l\\_a\\_00638/119630/Lost-in-the-Middle-How-Language-Models-Use-Long](https://direct.mit.edu/tac/article/doi/10.1162/tac_l_a_00638/119630/Lost-in-the-Middle-How-Language-Models-Use-Long).
21. Lopopolo, R. Harness engineering: leveraging Codex in an agent-first world. *OpenAI* 2026; <https://openai.com/index/harness-engineering/>.
22. Manning, C.D., Raghavan, P., and Schütze, H. *Introduction to Information Retrieval*. Cambridge University Press, Cambridge, UK 2008.
23. Mata v. Avianca Inc. 2023. 678 F. Supp. 3d 443 (S.D.N.Y. 2023).
24. Mayraz, O. CamoLeak: critical GitHub Copilot vulnerability leaks private source code. Legit Security. 2025; <https://www.legitsecurity.com/blog/camoleak-critical-github-copilot-vulnerability-leaks-private-source-code>.
25. Noda, A., Tacho, L., et al. Measuring developer productivity with the DX Core 4. DX. 2024; <https://getdx.com/research/measuring-developer-productivity-with-the-dx-core-4/>.
26. OWASP Foundation. OWASP Top 10 for LLM applications 2025. 2024; <https://genai.owasp.org/llm-top-10/>.
27. PromptArmor. Data exfiltration from Slack AI via indirect prompt injection. PromptArmor. 2024; <https://promptarmor.substack.com/p/data-exfiltration-from-slack-ai-via>.
28. Ray, S. Samsung bans ChatGPT among employees after sensitive code leak. *Forbes* 2023. (May 2); <https://www.forbes.com/sites/siladityaray/2023/05/02/samsung-bans-chatgpt-and-other-chatbots-for-employees-after-sensitive-code-leak/>.
29. Reddy, P. and Gujral, A.S. EchoLeak: the first real-world zero-click prompt injection exploit in a production LLM system. *Proceedings of the AAAI Symp.* 7(1), 2025, 303–311; <https://ojs.aaai.org/index.php/AAAI-SS/article/view/36899>.
30. Storey, M.-A. From technical debt to cognitive and intent debt: rethinking software health in the age of AI. *acmqueue* 24, 2 2026; <https://queue.acm.org/detail.cfm?id=3807966>.
31. Storey, M.-A., Houck, B., and Zimmermann, T. How developers and managers define and trade productivity for quality. *Proceedings of the 15th Intern. Conf. on Cooperative and Human Aspects of Software Engineering (CHASE)*, 2022, 26–35; <https://dl.acm.org/doi/10.1145/3528579.3529177>.
32. van Lamsweerde, A. Goal-oriented requirements engineering: a guided tour. *Proceedings of the Fifth IEEE Intern. Symp. on Requirements Engineering*, 2001, 249–262; <https://dl.acm.org/doi/10.5555/882477.883624>.

**Brian Houck** is a distinguished scientist at DX. His work combines large-scale telemetry analysis, field experiments, surveys, and qualitative research to uncover the technical, cultural, environmental, and organizational factors that shape developer productivity and well-being. He is best known for his work on the SPACE framework for developer productivity and for measuring and improving developer experience in large engineering organizations.

**Max Kanat-Alexander** is an executive distinguished engineer for developer experience at Capital One, where he leads technical strategy for AI-assisted developer experience for more than 14,000 technologists. He is the author of *Code Simplicity and Understanding Software* and has previously led code health initiatives at Google and the Bugzilla Project.

**Eirini Kalliamvakou** is a research advisor at GitHub and leads research that deciphers developers' motivations, needs, behavior, and how tools support them. Her insights shape product thinking, strategic storytelling, and leadership decisions across the company. She holds a Ph.D. in computer science from the University of Victoria and speaks about developer productivity, happiness, and the impact of AI on individuals, teams, and organizations.

**Margaret-Anne Storey** is a professor of computer science at the University of Victoria and a Canada research chair in human and social aspects of software engineering. She is coauthor of the SPACE framework and a leading researcher in developer experience (DevEx). Her research focuses on how developers and teams understand complex software systems and how tools, AI, and collaborative practices shape that understanding. Her recent work examines how generative AI is transforming software engineering by changing how understanding is created, shared, and maintained. She collaborates with industry partners including Microsoft and DX. She holds an honorary doctorate from Lund University.

**Nicole Forsgren** is the senior director of developer intelligence at Google. She is an expert in DevOps and the author of the Shingo Publication Award-winning book *Accelerate: The Science of Lean Software and DevOps*. Her work on technical practices and development has been published in industry and academic journals and is used to guide organizational transformations around the world.